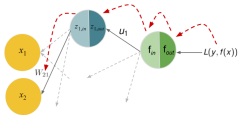


Advanced Artificial Intelligence and Machine Learning

Backpropagation



Learning goals

- Computational graphs and the chain rule as the mathematical basis of backpropagation
- The forward and backward pass, worked through a concrete example
- General backpropagation formalism via recursive application of the chain rule
- Why and how to initialize network weights and biases
- Optimization challenges in deep networks: ill-conditioning, local minima, saddle points, cliffs/exploding gradients

Chain Rule and Computational Graphs

CHAIN RULE OF CALCULUS

- The chain rule can be used to compute derivatives of the composition of two or more functions.
- Let $x \in \mathbb{R}^m$, $y \in \mathbb{R}^n$,
 $g : \mathbb{R}^m \rightarrow \mathbb{R}^n$ and $f : \mathbb{R}^n \rightarrow \mathbb{R}$.
- If $y = g(x)$ and $z = f(y)$, the chain rule yields:

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \cdot \frac{\partial y_j}{\partial x_i}$$

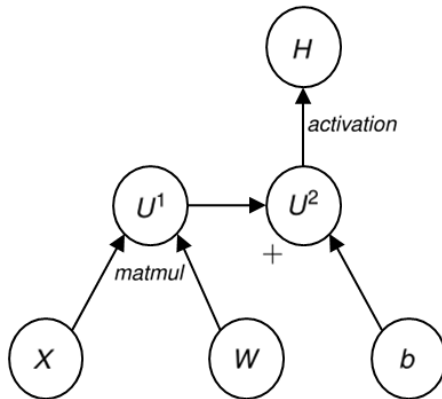
or, in vector notation:

$$\nabla_x z = \left(\frac{\partial y}{\partial x} \right)^\top \nabla_y z,$$

where $\frac{\partial y}{\partial x}$ is the $(n \times m)$ Jacobian matrix of g .

COMPUTATIONAL GRAPHS

- CGs are nested expressions, visualized as graphs.
- Each node is a variable, either an input or derived.
- Derived variables are functions applied to other variables.



source : Goodfellow et al. (2016)

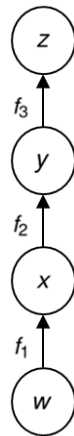
Figure: The computational graph for the expression $H = \sigma(XW + B)$ with activation function $\sigma(\cdot)$.

COMPUTATIONAL GRAPHS / 2

CHAIN RULE OF CALCULUS: EXAMPLE 1

- Suppose we have the following computational graph.
- To compute the derivative of $\frac{\partial z}{\partial w}$ we need to recursively apply the chain rule. That is:

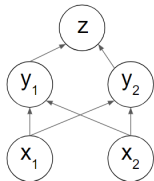
$$\begin{aligned}\frac{\partial z}{\partial w} &= \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x} \cdot \frac{\partial x}{\partial w} \\ &= f'_3(y) \cdot f'_2(x) \cdot f'_1(w) \\ &= f'_3(f_2(f_1(w))) \cdot f'_2(f_1(w)) \cdot f'_1(w)\end{aligned}$$



source : Goodfellow et al. (2016)

Figure: A computational graph, such that $x = f_1(w)$, $y = f_2(x)$ and

CHAIN RULE OF CALCULUS: EXAMPLE 2



To compute $\nabla_x z$, we apply the chain rule

- $\frac{\partial z}{\partial x_1} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_1} = \frac{\partial z}{\partial y_1} \frac{\partial y_1}{\partial x_1} + \frac{\partial z}{\partial y_2} \frac{\partial y_2}{\partial x_1}$
- $\frac{\partial z}{\partial x_2} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_2} = \frac{\partial z}{\partial y_1} \frac{\partial y_1}{\partial x_2} + \frac{\partial z}{\partial y_2} \frac{\partial y_2}{\partial x_2}$

Therefore, the gradient of z w.r.t x is

- $$\nabla_x z = \begin{bmatrix} \frac{\partial z}{\partial x_1} \\ \frac{\partial z}{\partial x_2} \end{bmatrix} = \underbrace{\begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_2}{\partial x_1} \\ \frac{\partial y_1}{\partial x_2} & \frac{\partial y_2}{\partial x_2} \end{bmatrix}}_{\left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}}\right)^\top} \underbrace{\begin{bmatrix} \frac{\partial z}{\partial y_1} \\ \frac{\partial z}{\partial y_2} \end{bmatrix}}_{\nabla_y z} = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}}\right)^\top \nabla_y z$$

COMPUTATIONAL GRAPH: NEURAL NET

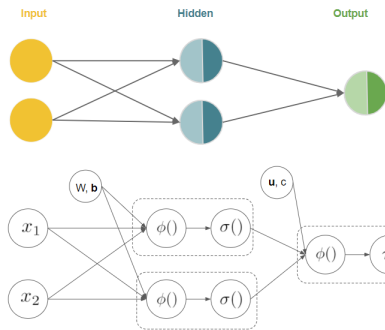


Figure: A neural network can be seen as a computational graph. ϕ is the weighted sum and σ and τ are the activations.

Note: In contrast to the top figure, the arrows in the computational graph below merely indicate **dependence**, not weights.

Basic Backpropagation 1

BACKPROPAGATION: BASIC IDEA

We would like to run ERM by GD on:

$$\mathcal{R}_{\text{emp}}(\theta) = \frac{1}{n} \sum_{i=1}^n L\left(y^{(i)}, f(x^{(i)} \mid \theta)\right)$$

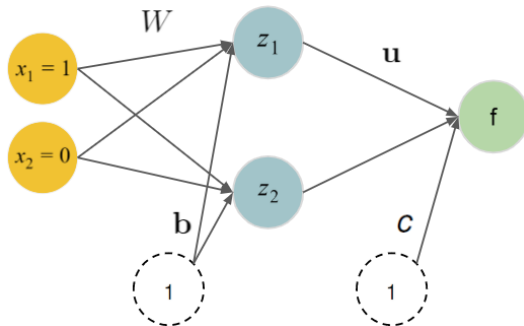
Backprop training of NNs runs in 2 alternating steps, for one x :

- ➊ **Forward pass:** Inputs flow through model to outputs. We then compute the observation loss. We covered that.
- ➋ **Backward pass:** Loss flows backwards to update weights so error is reduced, as in GD.

We will see: This is simply (S)GD in disguise, cleverly using the chain rule, so we can reuse a lot of intermediate results.

XOR EXAMPLE

- As activations (hidden and outputs) we use the logistic.
- We run one FP and BP on $x = (1, 0)^T$ with $y = 1$.
- We use L2 loss between 0-1 labels and the predicted probabilities.
This is a bit uncommon, but computations become simpler.

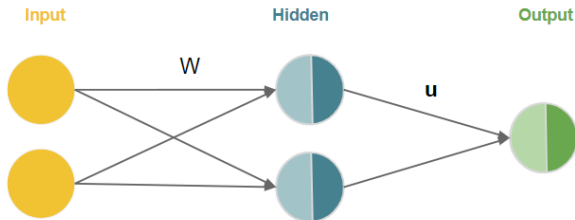


XOR EXAMPLE / 2

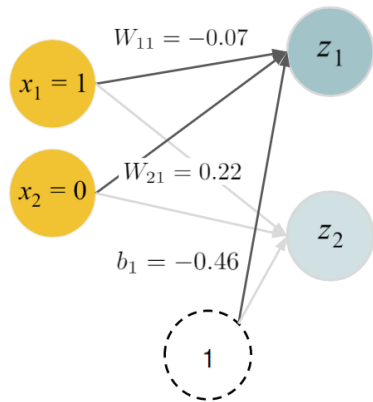
Note: We will only show rounded decimals.

FORWARD PASS

- We will divide the FP into four steps:
 - the inputs of z_j : $\mathbf{z}_{j,in}$
 - the activations of z_j : $\mathbf{z}_{j,out}$
 - the input of f : $\mathbf{f}_{in}$
 - and finally the activation of f : $\mathbf{f}_{out}$



FORWARD PASS / 2

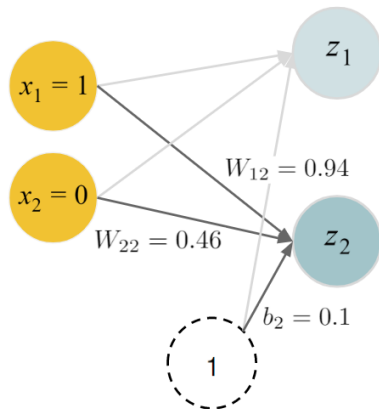


FORWARD PASS / 3

$$z_{1,in} = W_1^T x + b_1 = 1 \cdot (-0.07) + 0 \cdot 0.22 + 1 \cdot (-0.46) = -0.53$$

$$z_{1,out} = \sigma(z_{1,in}) = \frac{1}{1 + \exp(-(-0.53))} = 0.3705$$

FORWARD PASS / 4

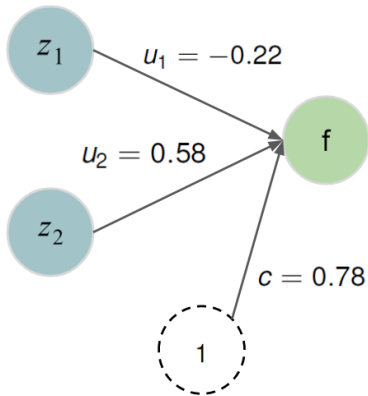


FORWARD PASS / 5

$$z_{2,in} = W_2^T x + b_2 = 1 \cdot 0.94 + 0 \cdot 0.46 + 1 \cdot 0.1 = 1.04$$

$$z_{2,out} = \sigma(z_{2,in}) = \frac{1}{1 + \exp(-1.04)} = 0.7389$$

FORWARD PASS / 6



FORWARD PASS / 7

$$f_{in} = \mathbf{u}^T \mathbf{z} + c = 0.3705 \cdot (-0.22) + 0.7389 \cdot 0.58 + 1 \cdot 0.78 = 1.1122$$

$$f_{out} = \tau(f_{in}) = \frac{1}{1 + \exp(-1.1122)} = 0.7525$$

FORWARD PASS / 8

- The FP predicted $f_{out} = 0.7525$
- Now, we compare the prediction $f_{out} = 0.7525$ and the true label $y = 1$ using the L2-loss:

$$\begin{aligned} L(y, f(x)) &= \frac{1}{2}(y - f(x^{(i)} | \theta))^2 = \frac{1}{2}(y - f_{out})^2 \\ &= \frac{1}{2}(1 - 0.7525)^2 = 0.0306 \end{aligned}$$

- The calculation of the gradient is performed backwards (starting from the output layer), so that results can be reused.

BACKWARD PASS

The main ingredients of the backward pass are:

- to reuse the results of the forward pass
(here: $z_{j,in}$, $z_{j,out}$, f_{in} , f_{out})
- reuse the **intermediate results** from the chain rule
- the derivative of the activations and some affine functions

BACKWARD PASS / 2

- Let's start to update u_1 . We recursively apply the chain rule:

$$\frac{\partial L(y, f(x))}{\partial u_1} = \frac{\partial L(y, f(x))}{\partial f_{out}} \cdot \frac{\partial f_{out}}{\partial f_{in}} \cdot \frac{\partial f_{in}}{\partial u_1}$$

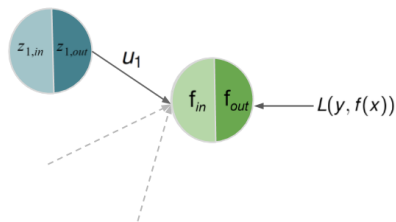
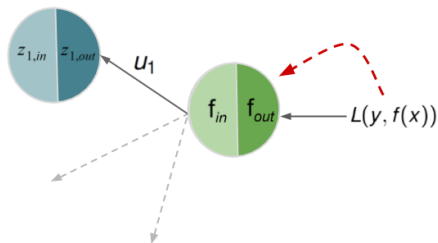


Figure: Snippet from our NN, with backward path for u_1 .

BACKWARD PASS / 3

- 1st step: The derivative of L2 is easy; we know f_{out} from FP.

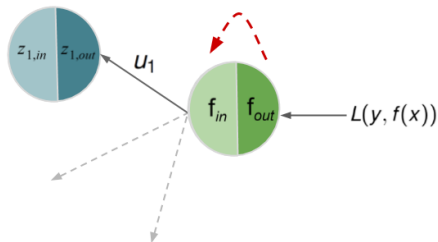
$$\begin{aligned}\frac{\partial L(y, f(x))}{\partial f_{out}} &= \frac{d}{df_{out}} \frac{1}{2} (y - f_{out})^2 = - \underbrace{(y - f_{out})}_{\hat{=} \text{residual}} \\ &= -(1 - 0.7525) = -0.2475\end{aligned}$$



BACKWARD PASS / 4

- 2nd step. $f_{out} = \sigma(f_{in})$, use rule for σ' , use f_{in} from FP.

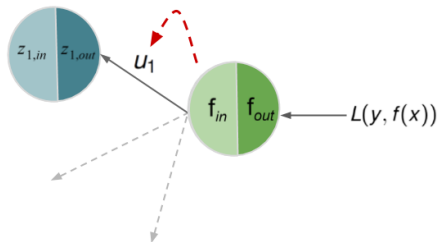
$$\begin{aligned}\frac{\partial f_{out}}{\partial f_{in}} &= \sigma(f_{in}) \cdot (1 - \sigma(f_{in})) \\ &= 0.7525 \cdot (1 - 0.7525) = 0.1862\end{aligned}$$



BACKWARD PASS / 5

- 3rd step. Derivative of the linear input is easy; use $z_{1,out}$ from FP.

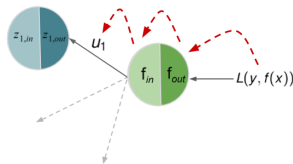
$$\frac{\partial f_{in}}{\partial u_1} = \frac{\partial(u_1 \cdot z_{1,out} + u_2 \cdot z_{2,out} + c \cdot 1)}{\partial u_1} = z_{1,out} = 0.3705$$



BACKWARD PASS / 6

- Plug it together:

$$\begin{aligned}\frac{\partial L(y, f(x))}{\partial u_1} &= \frac{\partial L(y, f(x))}{\partial f_{out}} \cdot \frac{\partial f_{out}}{\partial f_{in}} \cdot \frac{\partial f_{in}}{\partial u_1} \\ &= -0.2475 \cdot 0.1862 \cdot 0.3705 = -0.0171\end{aligned}$$



BACKWARD PASS / 7

- With LR $\alpha = 0.5$:

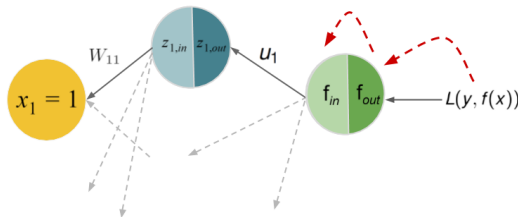
$$\begin{aligned}u_1^{[t+1]} &= u_1^{[t]} - \alpha \cdot \frac{\partial L(y, f(\mathbf{x}))}{\partial u_1} \\&= -0.22 - 0.5 \cdot (-0.0171) = -0.2115\end{aligned}$$

BACKWARD PASS / 8

- Now for W_{11} :

$$\frac{\partial L(y, f(x))}{\partial W_{11}} = \frac{\partial L(y, f(x))}{\partial f_{out}} \cdot \frac{\partial f_{out}}{\partial f_{in}} \cdot \frac{\partial f_{in}}{\partial z_{1,out}} \cdot \frac{\partial z_{1,out}}{\partial z_{1,in}} \cdot \frac{\partial z_{1,in}}{\partial W_{11}}$$

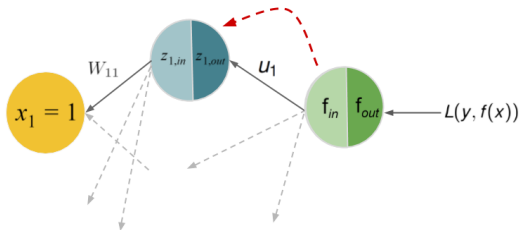
- We know $\frac{\partial L(y, f(x))}{\partial f_{out}}$ and $\frac{\partial f_{out}}{\partial f_{in}}$ from BP for u_1 .



BACKWARD PASS / 9

- $f_{in} = u_1 \cdot z_{1,out} + u_2 \cdot z_{2,out} + c \cdot 1$ is linear, easy and we know u_1 :

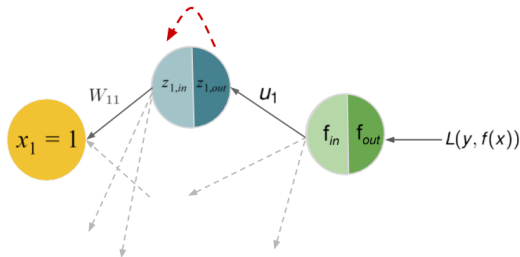
$$\frac{\partial f_{in}}{\partial z_{1,out}} = u_1 = -0.22$$



BACKWARD PASS / 10

- Next. Use rule for σ' and FP results:

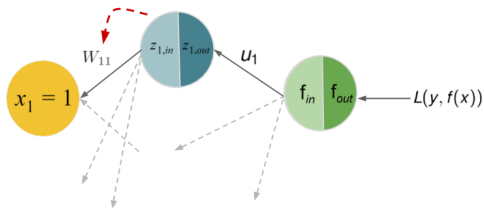
$$\begin{aligned}\frac{\partial z_{1,out}}{\partial z_{1,in}} &= \sigma(z_{1,in}) \cdot (1 - \sigma(z_{1,in})) \\ &= 0.3705 \cdot (1 - 0.3705) = 0.2332\end{aligned}$$



BACKWARD PASS / 11

- $z_{1,in} = x_1 \cdot W_{11} + x_2 \cdot W_{21} + b_1 \cdot 1$ is linear and depends on inputs:

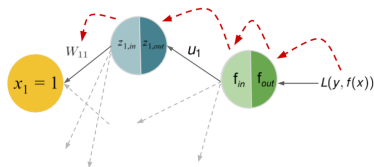
$$\frac{\partial z_{1,in}}{\partial W_{11}} = x_1 = 1$$



BACKWARD PASS / 12

- Plugging together:

$$\begin{aligned}\frac{\partial L(y, f(x))}{\partial W_{11}} &= \frac{\partial L(y, f(x))}{\partial f_{out}} \cdot \frac{\partial f_{out}}{\partial f_{in}} \cdot \frac{\partial f_{in}}{\partial z_{1,out}} \cdot \frac{\partial z_{1,out}}{\partial z_{1,in}} \cdot \frac{\partial z_{1,in}}{\partial W_{11}} \\ &= (-0.2475) \cdot 0.1862 \cdot (-0.22) \cdot 0.2332 \cdot 1 \\ &= 0.0024\end{aligned}$$



BACKWARD PASS / 13

- Full SGD update:

$$\begin{aligned}W_{11}^{[t+1]} &= W_{11}^{[t]} - \alpha \cdot \frac{\partial L(y, f(x))}{\partial W_{11}} \\&= -0.07 - 0.5 \cdot 0.0024 = -0.0712\end{aligned}$$

RESULT

- We can do this for all weights:

$$\mathbf{w} = \begin{pmatrix} -0.0712 & 0.9426 \\ 0.22 & 0.46 \end{pmatrix}, \mathbf{b} = \begin{pmatrix} -0.4612 \\ 0.1026 \end{pmatrix},$$

$$\mathbf{u} = \begin{pmatrix} -0.2115 \\ 0.5970 \end{pmatrix} \text{ and } c = 0.8030.$$

- Yields $f(x | \boldsymbol{\theta}^{[t+1]}) = 0.7615$ and loss $\frac{1}{2}(1 - 0.7615)^2 = 0.0284$.
- Before, we had $f(x | \boldsymbol{\theta}^{[t]}) = 0.7525$ and higher loss 0.0306.

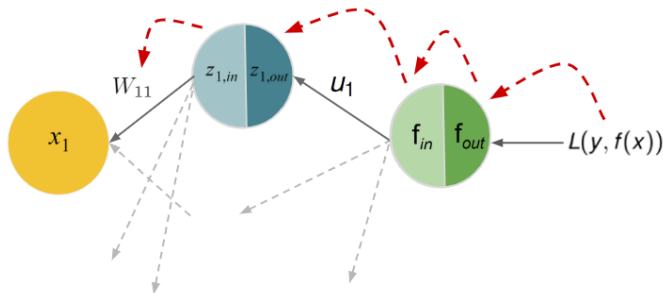
Now rinse and repeat. This was one training iter, we do thousands.

Basic Backpropagation 2

BACKWARD COMPUTATION AND CACHING

In the XOR example, we computed:

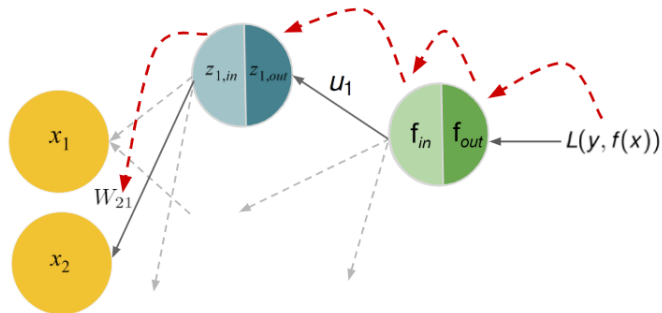
$$\frac{\partial L(y, f(x))}{\partial W_{11}} = \frac{\partial L(y, f(x))}{\partial f_{out}} \cdot \frac{\partial f_{out}}{\partial f_{in}} \cdot \frac{\partial f_{in}}{\partial z_{1,out}} \cdot \frac{\partial z_{1,out}}{\partial z_{1,in}} \cdot \frac{\partial z_{1,in}}{\partial W_{11}}$$



BACKWARD COMPUTATION AND CACHING

Next, let us compute:

$$\frac{\partial L(y, f(x))}{\partial W_{21}} = \frac{\partial L(y, f(x))}{\partial f_{out}} \cdot \frac{\partial f_{out}}{\partial f_{in}} \cdot \frac{\partial f_{in}}{\partial z_{1,out}} \cdot \frac{\partial z_{1,out}}{\partial z_{1,in}} \cdot \frac{\partial z_{1,in}}{\partial W_{21}}$$



BACKWARD COMPUTATION AND CACHING

- Examining the two expressions:

$$\frac{\partial L(y, f(x))}{\partial W_{11}} = \frac{\partial L(y, f(x))}{\partial f_{out}} \cdot \frac{\partial f_{out}}{\partial f_{in}} \cdot \frac{\partial f_{in}}{\partial z_{1,out}} \cdot \frac{\partial z_{1,out}}{\partial z_{1,in}} \cdot \frac{\partial z_{1,in}}{\partial W_{11}}$$

$$\frac{\partial L(y, f(x))}{\partial W_{21}} = \frac{\partial L(y, f(x))}{\partial f_{out}} \cdot \frac{\partial f_{out}}{\partial f_{in}} \cdot \frac{\partial f_{in}}{\partial z_{1,out}} \cdot \frac{\partial z_{1,out}}{\partial z_{1,in}} \cdot \frac{\partial z_{1,in}}{\partial W_{21}}$$

- Significant overlap / redundancy in the two expressions.
- Again:** Let's call this subexpression δ_1 and cache it.

$$\delta_1 = \frac{\partial L(y, f(x))}{\partial z_{1,in}} = \frac{\partial L(y, f(x))}{\partial f_{out}} \cdot \frac{\partial f_{out}}{\partial f_{in}} \cdot \frac{\partial f_{in}}{\partial z_{1,out}} \cdot \frac{\partial z_{1,out}}{\partial z_{1,in}}$$

$$\frac{\partial L(y, f(x))}{\partial W_{11}} = \delta_1 \cdot \frac{\partial z_{1,in}}{\partial W_{11}} \quad \text{and} \quad \frac{\partial L(y, f(x))}{\partial W_{21}} = \delta_1 \cdot \frac{\partial z_{1,in}}{\partial W_{21}}$$

- δ_1 can also be seen as an **error signal** that represents how much the loss / changes when the input z_1 changes

BACKPROP: RECURSION

- Let us now derive a general formulation of backprop.
- The neurons in layers $i - 1$, i and $i + 1$ are indexed by j , k and m , respectively.
- The output layer will be referred to as layer O.

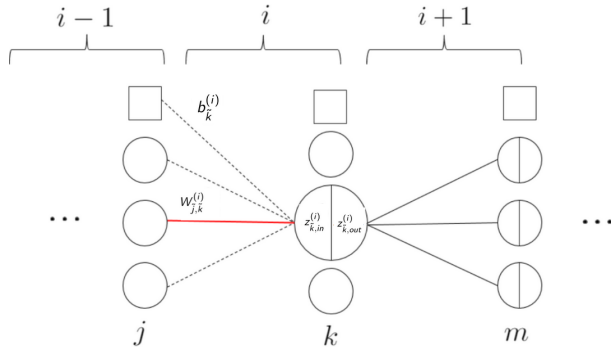
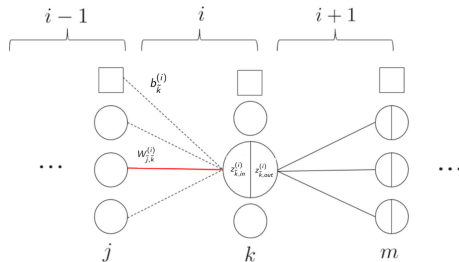


Figure: (Hallström, 2016)

BACKPROP: RECURSION



- Let $\delta_{\tilde{k}}^{(i)}$ (also: error signal) for a neuron $\tilde{k}$ in layer i represent how much the loss L changes when the input $z_{\tilde{k},in}^{(i)}$ changes:

$$\delta_{\tilde{k}}^{(i)} = \frac{\partial L}{\partial z_{\tilde{k},in}^{(i)}} = \frac{\partial L}{\partial z_{\tilde{k},out}^{(i)}} \frac{\partial z_{\tilde{k},out}^{(i)}}{\partial z_{\tilde{k},in}^{(i)}} = \sum_m \left(\frac{\partial L}{\partial z_{m,in}^{(i+1)}} \frac{\partial z_{m,in}^{(i+1)}}{\partial z_{\tilde{k},out}^{(i)}} \right) \frac{\partial z_{\tilde{k},out}^{(i)}}{\partial z_{\tilde{k},in}^{(i)}}$$

BACKPROP: RECURSION / 2

- Note: The sum in the expression above is over all the neurons in layer $i + 1$. This is simply an application of the (multivariate) chain rule.

BACKPROP: RECURSION

Using

$$\begin{aligned}z_{\tilde{k},out}^{(i)} &= \sigma(z_{\tilde{k},in}^{(i)}) \\z_{m,in}^{(i+1)} &= \sum_k W_{k,m}^{(i+1)} z_{k,out}^{(i)} + b_m^{(i+1)}\end{aligned}$$

we get:

$$\begin{aligned}\delta_{\tilde{k}}^{(i)} &= \sum_m \left(\frac{\partial L}{\partial z_{m,in}^{(i+1)}} \frac{\partial z_{m,in}^{(i+1)}}{\partial z_{\tilde{k},out}^{(i)}} \right) \frac{\partial z_{\tilde{k},out}^{(i)}}{\partial z_{\tilde{k},in}^{(i)}} \\&= \sum_m \left(\frac{\partial L}{\partial z_{m,in}^{(i+1)}} \frac{\partial \left(\sum_k W_{k,m}^{(i+1)} z_{k,out}^{(i)} + b_m^{(i+1)} \right)}{\partial z_{\tilde{k},out}^{(i)}} \right) \frac{\partial \sigma(z_{\tilde{k},in}^{(i)})}{\partial z_{\tilde{k},in}^{(i)}} \\&= \sum_m \left(\frac{\partial L}{\partial z_{m,in}^{(i+1)}} W_{\tilde{k},m}^{(i+1)} \right) \sigma'(z_{\tilde{k},in}^{(i)}) = \sum_m \left(\delta_{\tilde{k}}^{(i+1)} W_{\tilde{k},m}^{(i+1)} \right) \sigma'(z_{\tilde{k},in}^{(i)})\end{aligned}$$

Therefore, we now have a **recursive definition** for the error signal of a neuron in layer i in terms of the error signals of the neurons in layer $i + 1$ and, by extension, layers $\{i+2, i+3 \dots, O\}$!

BACKPROP: RECURSION

- Given the error signal $\delta_{\tilde{k}}^{(i)}$ of neuron $\tilde{k}$ in layer i , the derivative of loss L w.r.t. to the weight $W_{j,\tilde{k}}$ is simply:

$$\frac{\partial L}{\partial W_{j,\tilde{k}}^{(i)}} = \frac{\partial L}{\partial z_{\tilde{k},in}^{(i)}} \frac{\partial z_{\tilde{k},in}^{(i)}}{\partial W_{j,\tilde{k}}^{(i)}} = \delta_{\tilde{k}}^{(i)} z_{j,out}^{(i-1)}$$

because $z_{\tilde{k},in}^{(i)} = \sum_j W_{j,\tilde{k}}^{(i)} z_{j,out}^{(i-1)} + b_{\tilde{k}}^{(i)}$

- Similarly, the derivative of loss L w.r.t. bias $b_{\tilde{k}}^{(i)}$ is:

$$\frac{\partial L}{\partial b_{\tilde{k}}^{(i)}} = \frac{\partial L}{\partial z_{\tilde{k},in}^{(i)}} \frac{\partial z_{\tilde{k},in}^{(i)}}{\partial b_{\tilde{k}}^{(i)}} = \delta_{\tilde{k}}^{(i)}$$

BACKPROP: RECURSION

- It is not hard to show that the error signal δ^i for an entire layer i is ($\odot$ = element-wise product):
 - $\delta^{(O)} = \nabla_{f_{out}} L \odot \tau'(f_{in})$
 - $\delta^{(i)} = W^{(i+1)} \delta^{(i+1)} \odot \sigma'(z_{in}^{(i)})$
- Therefore, backpropagation works by computing and storing the error signals **backwards**. That is, starting at the output layer and ending at the first hidden layer. This way, the error signals of later layers **propagate backwards** to the earlier layers.
- The derivative of the loss L w.r.t. a given weight is computed efficiently by plugging in the cached error signals, thereby avoiding expensive and redundant computations.

REFERENCES



Hallström, E. (2016, December 4). *Backpropagation from the beginning*. Medium. <https://medium.com/@erikhallstrm/backpropagation-from-the-beginning-77356edf427d>

Network Initializations

PRACTICAL INITIALIZATION

- The weights (and biases) of a neural network must be assigned some initial values before training can begin.
- The choice of the initial weights (and biases) is crucial as it determines whether an optimization algorithm converges, how fast and whether to a point with high or low risk.
- Initialization strategies to achieve "nice" properties are difficult to find, because there is no good understanding which properties are preserved under which circumstances.
- In the following we separate between the initialization of weights and biases.

WEIGHT INITIALIZATION

- It is important to initialize the weights randomly in order to "break symmetry". If two neurons (with the same activation function in a fully connected network) are connected to the same inputs and have the same initial weights, then both neurons will have the same gradient update in a given iteration and they will end up learning the same features.
- Furthermore, the initial weights should not be too large, because this might result in an explosion of weights or high sensitivity to changes in the input.
- Weights are typically drawn from a uniform distribution or a Gaussian centered at 0 with a small variance.
- Centering the initial weights around 0 can be seen as a form of regularization and it imposes that it is more likely that units do not interact with each other than they do interact.

WEIGHT INITIALIZATION / 2

- Two common initialization strategies for weights are the 'Glorot initialization' and 'He initialization' which tune the variance of these distributions based on the topology of the network.
- **Glorot initialization** suggests to sample each weight of a fully connected layer with m inputs and n outputs from a uniform distribution

$$w_{j,k} \sim U\left(-\sqrt{\frac{6}{m+n}}, \sqrt{\frac{6}{m+n}}\right)$$

The strategy is derived from the assumption that the network consists only of a chain of matrix multiplications with no nonlinearities.

WEIGHT INITIALIZATION / 3

- **He initialization** is especially useful for neural networks with ReLU activations. Each weight of a fully connected layer with m inputs is sampled from a Gaussian distribution

$$w_{j,k} \sim N\left(0, \frac{2}{m}\right)$$

The underlying derivation can be found in He et. al. (2015).

- Since the initialization strategies of Glorot and He depend on the layer sizes, the initial weights for large layer sizes can become extremely small.
- Another strategy is to treat the weights as hyperparameters that can be optimized by hyperparameter search algorithms. This can be computationally costly.

WEIGHT INITIALIZATION: EXAMPLE

- We use a spiral planar data set to compare the following strategies:
Zero initialization, random initialization (samples from $N(0, 1 \cdot 10^{-4})$) and He initialization.
- For each strategy, a neural network with one hidden layer with 100 units, ReLU activation and Gradient Descent as optimizer was used.

WEIGHT INITIALIZATION: EXAMPLE / 2

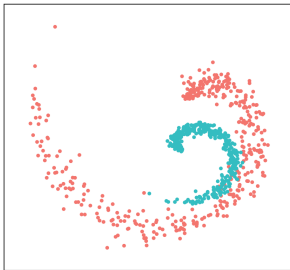


Figure: Simulated spiral planar data set with two classes (Ghatak, 2019).

WEIGHT INITIALIZATION: EXAMPLE / 3

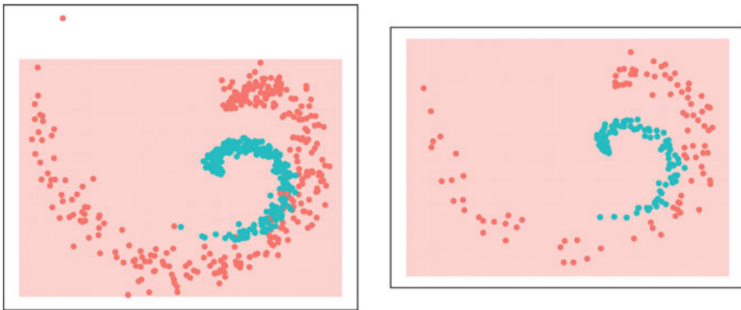


Figure: Decision boundary with zero initialization on the training data set (left) and the testing data set (right). The zero initialization does not break symmetry and the complexity of the network reduces to that of a single neuron (Ghatak, 2019).

WEIGHT INITIALIZATION: EXAMPLE / 4

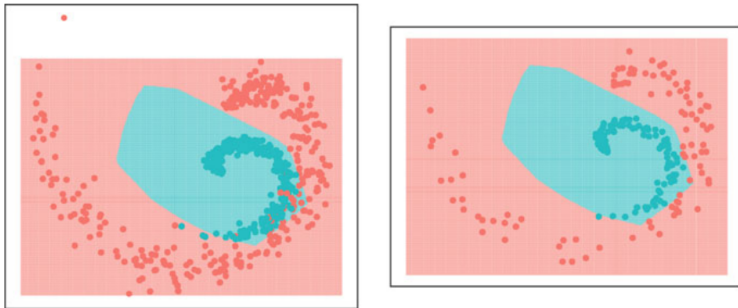


Figure: Decision boundary with random initialization ($N(0, 1 \cdot 10^{-4})$) on the training data set (left) and the testing data set (right) (Ghatak, 2019).

WEIGHT INITIALIZATION: EXAMPLE / 5

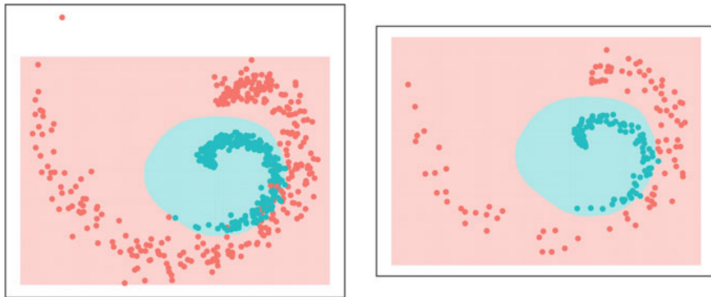


Figure: Decision boundary with He initialization on the training data set (left) and the testing data set (right) (Ghatak, 2019).

BIAS INITIALIZATION

- Typically, we set the biases for each unit to heuristically chosen constants.
- Setting the biases to zero is compatible with most weight initialization schemes as the schemes expect a small bias.
- However, deviations from 0 can be made individually, for example, in order to obtain the right marginal statistics of the output unit or to avoid causing too much saturation at the initialization.
- For details see Goodfellow et. al (2016).

REFERENCES



Ghatak, A. (2019). *Deep learning with R*. Springer Singapore.



Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

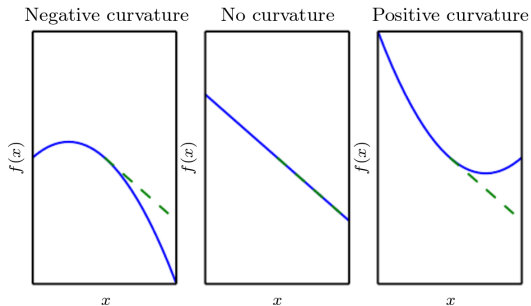
CHALLENGES IN OPTIMIZATION

- In this section, we summarize several of the most prominent challenges regarding training of deep neural networks.
- Traditionally, machine learning ensures that the optimization problem is convex by carefully designing the objective function and constraints. But for neural networks we are confronted with the general nonconvex case.
- Furthermore, we will see in this section that even convex optimization is not without its complications.

III-Conditioning

EFFECTS OF CURVATURE

Intuitively, the curvature of a function determines the outcome of a GD step. .



Source: Goodfellow *et al.*, (2016), ch. 4

Figure: Quadratic objective function $f(x)$ with various curvatures. The dashed line indicates the first order Taylor approximation based on the gradient information alone. Left: With negative curvature, the cost function decreases faster than the gradient predicts; Middle: With no curvature, the gradient predicts the decrease correctly; Right: With positive curvature, the function decreases more slowly than expected and begins to increase.

SECOND DERIVATIVE AND CURVATURE

To understand better how the curvature of a function influences the outcome of a gradient descent step, let us recall how curvature is described mathematically:

- The second derivative corresponds to the curvature of the graph of a function.
- The **Hessian** matrix of a function $\mathcal{R}(\boldsymbol{\theta}) : \mathbb{R}^m \rightarrow \mathbb{R}$ is the matrix of second-order partial derivatives

$$H_{ij} = \frac{\partial^2}{\partial \theta_i \partial \theta_j} \mathcal{R}(\boldsymbol{\theta}).$$

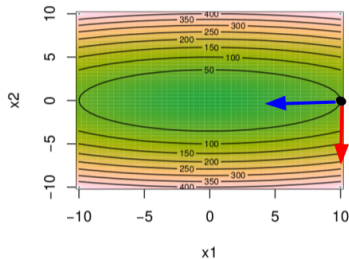
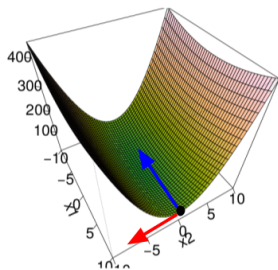
SECOND DERIVATIVE AND CURVATURE

/ 2

- The second derivative in a direction d , with $\|d\| = 1$, is given by $d^T H d$.
- What is the direction of the highest curvature (red direction), and what is the direction of the lowest curvature (blue)?

SECOND DERIVATIVE AND CURVATURE

/ 3



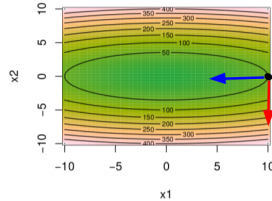
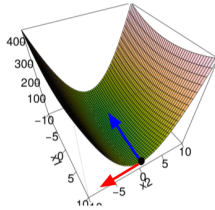
SECOND DERIVATIVE AND CURVATURE

/ 4

- Since H is real and symmetric (why?), eigendecomposition yields $H = V \text{diag}(\lambda) V^{-1}$ with V and λ collecting eigenvectors and eigenvalues, respectively.
- It can be shown, that the eigenvector $\mathbf{v}_{\max}$ with the max. eigenvalue $\lambda_{\max}$ points into the direction of highest curvature ($\mathbf{v}_{\max}^T H \mathbf{v}_{\max} = \lambda_{\max}$), while the eigenvector $\mathbf{v}_{\min}$ with the min. eigenvalue $\lambda_{\min}$ points into the direction of least curvature.

SECOND DERIVATIVE AND CURVATURE

/ 5



SECOND DERIVATIVE AND CURVATURE

/ 6

- At a stationary point θ , where the gradient is 0, we can examine the eigenvalues of the Hessian to determine whether the θ is a local maximum, minimum or saddle point:

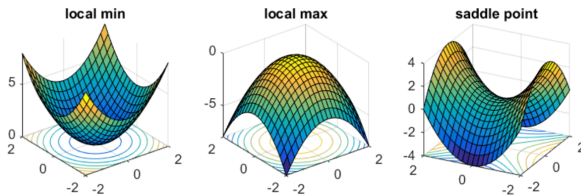
$\forall i : \lambda_i > 0$ (H positive definite at θ) $\Rightarrow$ minimum at θ

$\forall i : \lambda_i < 0$ (H negative definite at θ) $\Rightarrow$ maximum at θ

$\exists i : \lambda_i < 0 \wedge \exists j : \lambda_j > 0$ (H indefinit at θ) $\Rightarrow$ saddle point at θ

SECOND DERIVATIVE AND CURVATURE

/ 7

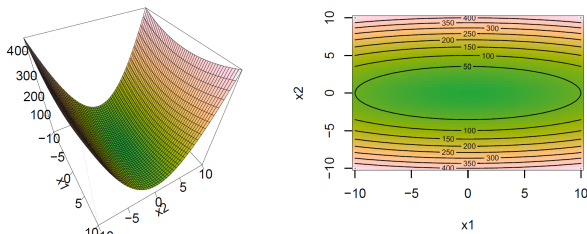


Credit: Rong Ge (2016)

ILL-CONDITIONED HESSIAN MATRIX

The condition number of a symmetric matrix $\mathbf{A}$ is given by the ratio of its min/max eigenvalues $\kappa(\mathbf{A}) = \frac{|\lambda_{\max}|}{|\lambda_{\min}|}$. A matrix is called ill-conditioned, if the condition number $\kappa(\mathbf{A})$ is very high.

An **ill-conditioned** Hessian matrix means that the ratio of max. / min. curvature is high, as in the example below:



CURVATURE AND STEP-SIZE IN GD

What does it mean for gradient descent if the Hessian is ill-conditioned?

- Let us consider the second-order Taylor approximation as a local approximation of the of $\mathcal{R}$ around a current point θ^0 (with gradient g)

$$\mathcal{R}(\theta) \approx T_2 f(\theta, \theta^0) := \mathcal{R}(\theta^0) + (\theta - \theta^0)^\top g + \frac{1}{2}(\theta - \theta^0)^\top H (\theta - \theta^0)$$

- Furthermore, Taylor's theorem states (proof in Koenigsberger (1997), p. 68)

$$\lim_{\theta \rightarrow \theta^0} \frac{\mathcal{R}(\theta) - T_2 f(\theta, \theta^0)}{\|\theta - \theta^0\|^2} = 0$$

CURVATURE AND STEP-SIZE IN GD / 2

- One GD step with a learning rate α yields new parameters $\theta^0 - \alpha g$ and a new approximated loss value

$$\mathcal{R}(\theta^0 - \alpha g) \approx \mathcal{R}(\theta^0) - \alpha g^\top g + \frac{1}{2} \alpha^2 g^\top H g.$$

- Theoretically, if $g^\top H g$ is positive, we can solve the equation above for the optimal step size which corresponds to

$$\alpha^* = \frac{g^\top g}{g^\top H g}.$$

CURVATURE AND STEP-SIZE IN GD / 3

- Let us assume the gradient g points into the direction of $\mathbf{v}_{\max}$ (i.e. the direction of highest curvature), the optimal step size is given by

$$\alpha^* = \frac{g^\top g}{g^\top H g} = \frac{g^\top g}{\lambda_{\max} g^\top g} = \frac{1}{\lambda_{\max}},$$

which is very small. Choosing a too large step-size is bad, as it will make us “overshoot” the stationary point.

- If, on the other hand, g points into the direction of the lowest curvature, the optimal step size is

$$\alpha^* = \frac{1}{\lambda_{\min}},$$

which corresponds to the largest possible optimal step-size.

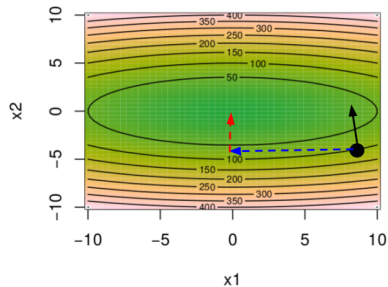
CURVATURE AND STEP-SIZE IN GD / 4

- We summarize: We want to perform big steps in directions of low curvature, but small steps in directions of high curvature.

CURVATURE AND STEP-SIZE IN GD / 5

- But what if the gradient does not point into the direction of one of the eigenvectors?
- Let us consider the 2-dimensional case: We can decompose the direction of $\mathbf{g}$ (black) into the two eigenvectors $\mathbf{v}_{\max}$ and $\mathbf{v}_{\min}$
- It would be optimal to perform a **big** step into the direction of the smallest curvature $\mathbf{v}_{\min}$, but a **small** step into the direction of $\mathbf{v}_{\max}$, but the gradient points into a completely different direction.

CURVATURE AND STEP-SIZE IN GD / 6

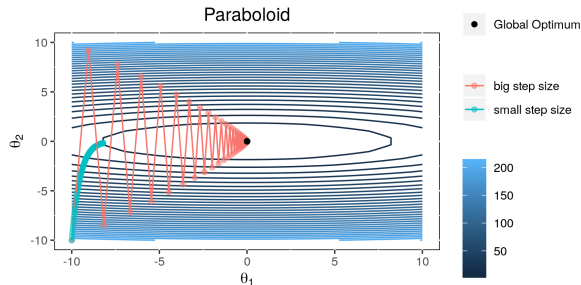


ILL-CONDITIONING

- GD is unaware of large differences in curvature, and can only walk into the direction of the gradient.
- Choosing a too large step-size will then cause the descent direction change frequently (“jumping around”).
- α needs to be small enough, which results in a low progress.

ILL-CONDITIONING / 2

- This effect is more severe, if a Hessian has a poor condition number, i.e. the ratio between lowest and highest curvature is large; gradient descent will perform poorly.



ILL-CONDITIONING / 3

Figure: The contour lines show a quadratic risk function with a poorly conditioned Hessian matrix. The plot shows the progress of gradient descent with a small step-size vs. larger step-size. In both cases, convergence to the global optimum is rather slow.

- In the worst case, ill-conditioning of the Hessian matrix and a too big step-size will cause the risk to increase

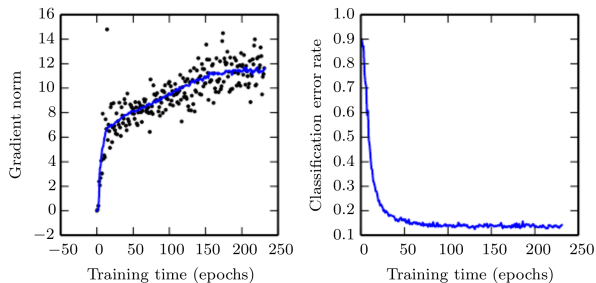
$$\mathcal{R}(\boldsymbol{\theta}^0 - \alpha \mathbf{g}) \approx \mathcal{R}(\boldsymbol{\theta}^0) - \alpha \mathbf{g}^\top \mathbf{g} + \frac{1}{2} \alpha^2 \mathbf{g}^\top \mathbf{H} \mathbf{g},$$

which happens if

$$\frac{1}{2} \alpha^2 \mathbf{g}^\top \mathbf{H} \mathbf{g} > \alpha \mathbf{g}^\top \mathbf{g}.$$

ILL-CONDITIONING / 4

- To determine whether ill-conditioning is detrimental to the training, the squared gradient norm $g^\top g$ and the risk can be monitored.



Source: Goodfellow, ch. 6

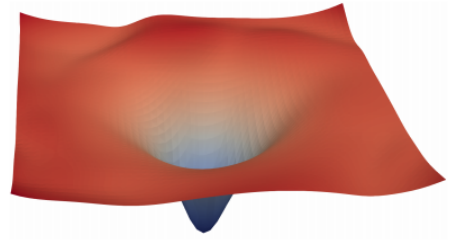
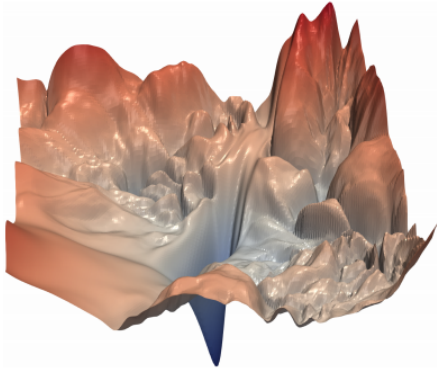
ILL-CONDITIONING / 5

- Gradient norms **increase** over time, showing that the training process is not converging to a stationary point $g = 0$.
- At the same time, we observe that the risk is approx. constant, but the gradient norm increases

$$\underbrace{\mathcal{R}(\theta^0 - \alpha g)}_{\text{approx. constant}} \approx \mathcal{R}(\theta^0) - \underbrace{\alpha g^\top g}_{\text{increase}} + \frac{1}{2} \underbrace{\alpha^2 g^\top H g}_{\rightarrow \text{increase}}.$$

Local Minima

UNIMODAL VS. MULTIMODAL LOSS SURFACES

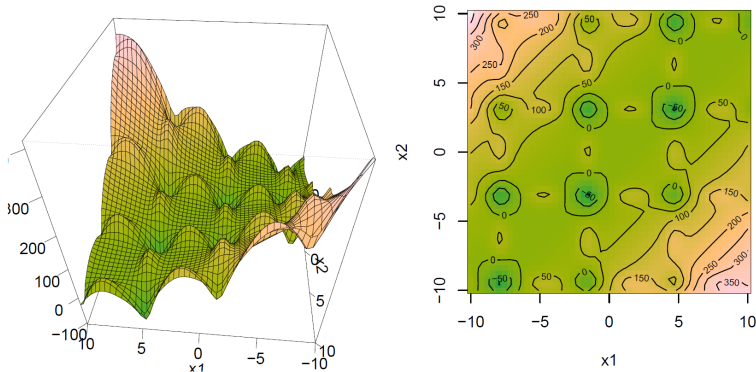


UNIMODAL VS. MULTIMODAL LOSS SURFACES / 2

Figure: Left: Multimodal loss surface with saddle points; Right: (Nearly) unimodal loss surface (Hao Li et al. (2017))

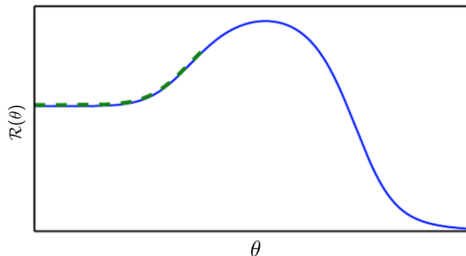
MULTIMODAL FUNCTION

Potential snippet from a loss surface of a deep neural network with many local minima:



ONLY LOCALLY OPTIMAL MOVES

- If the training algorithm makes only *locally* optimal moves (as in gradient descent), it may move away from regions of *much* lower cost.



Source: Goodfellow, Ch. 8

- In the figure above, initializing the parameter on the "wrong" side of the hill will result in suboptimal performance.
- In higher dimensions, however, it may be possible for gradient descent to go around the hill but such a trajectory might be very long and result in excessive training time.

LOCAL MINIMA

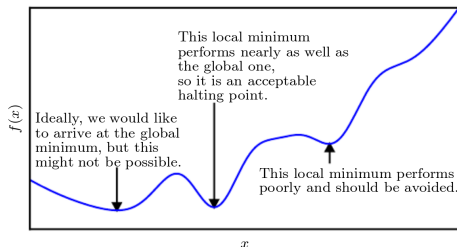
Weight space symmetry:

- If we swap incoming weight vectors for neuron i and j and do the same for the outgoing weights, modelled function stays unchanged.
⇒ with n hidden units and one hidden layer there are $n!$ networks with the same empirical risk
- If we multiply incoming weights of a ReLU neuron with β and outgoing with $1/\beta$ the modelled function stays unchanged.

⇒ The empirical risk of a NN can have very many minima with equivalent empirical risk.

LOCAL MINIMA / 2

- In practice only local minima with a high value compared to the global minimum are problematic.



Source: Goodfellow, Ch. 4

- Current literature suspects that most local minima have low empirical risk.

Saddle Points

SADDLE POINTS

- In optimization we look for areas with zero gradient.
- A variant of zero gradient areas are saddle points.
- For the empirical risk $\mathcal{R}$ of a neural network, the expected ratio of the number of saddle points to local minima typically grows exponentially with m

$$\mathcal{R} : \mathbb{R}^m \rightarrow \mathbb{R}$$

In other words: Networks with more parameters (deeper networks or larger layers) exhibit a lot more saddle points than local minima.

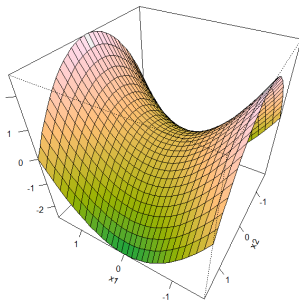
- Why is that?
- The Hessian at a local minimum has only positive eigenvalues. At a saddle point it is a mixture of positive and negative eigenvalues.

SADDLE POINTS / 2

- Imagine the sign of each eigenvalue is generated by coin flipping:
 - In a single dimension, it is easy to obtain a local minimum (e.g. “head” means positive eigenvalue).
 - In an m -dimensional space, it is exponentially unlikely that all m coin tosses will be head.
- A property of many random functions is that eigenvalues of the Hessian become more likely to be positive in regions of lower cost.
- For the coin flipping example, this means we are more likely to have heads m times if we are at a critical point with low cost.
- That means in particular that local minima are much more likely to have low cost than high cost and critical points with high cost are far more likely to be saddle points.
- See Dauphin et al. (2014) for a more detailed investigation.

SADDLE POINTS: EXAMPLE

$$f(x_1, x_2) = x_1^2 - x_2^2$$

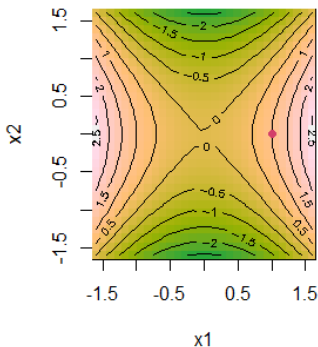
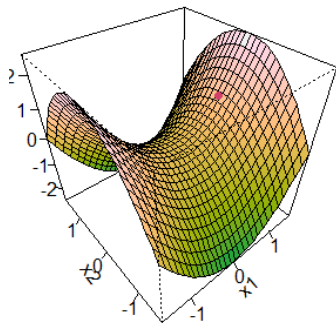


- Along x_1 , the function curves upwards (eigenvector of the Hessian with positive eigenvalue). Along x_2 , the function curves downwards (eigenvector of the Hessian with negative eigenvalue).

SADDLE POINTS

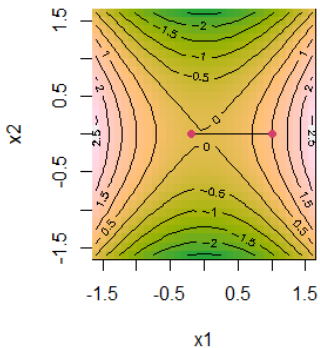
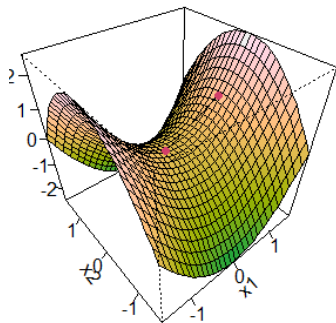
- So how do saddle points impair optimization?
- First-order algorithms that use only gradient information **might** get stuck in saddle points.
- Second-order algorithms experience even greater problems when dealing with saddle points. Newton's method for example actively searches for a region with zero gradient. That might be another reason why second-order methods have not succeeded in replacing gradient descent for neural network training.

EXAMPLE: SADDLE POINT WITH GD



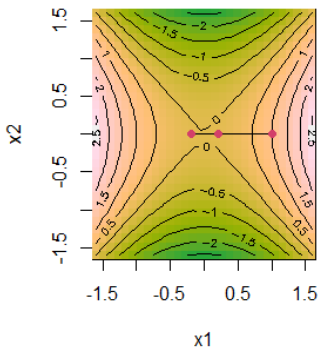
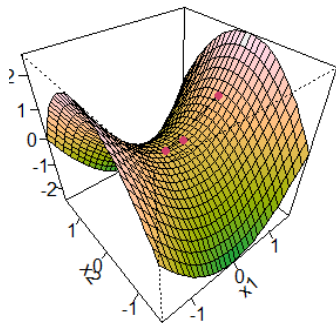
Red dot: Starting location

EXAMPLE: SADDLE POINT WITH GD



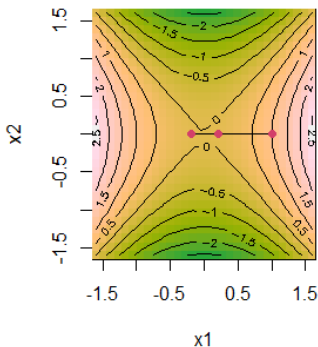
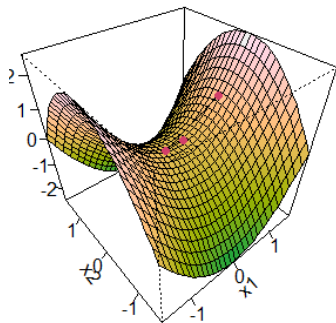
First step...

EXAMPLE: SADDLE POINT WITH GD



...second step...

EXAMPLE: SADDLE POINT WITH GD



...tenth step got stuck and cannot escape the saddle point!

Cliffs and Exploding Gradients

CLIFFS AND EXPLODING GRADIENTS

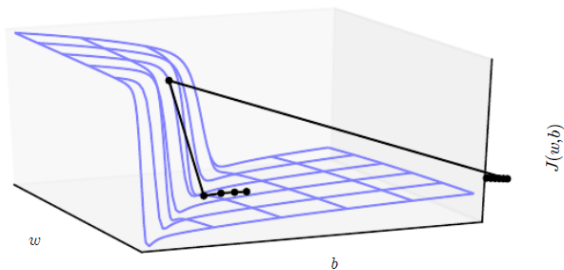
- As a result from the multiplication of several parameters, the empirical risk for highly nonlinear deep neural networks often contain sharp nonlinearities.
 - That may result in very high derivatives in some places.
 - As the parameters get close to such cliff regions, a gradient descent update can catapult the parameters very far.
 - Such an occurrence can lead to losing most of the optimization work that had been done.
- However, serious consequences can be easily avoided using a technique called **gradient clipping**.
- The gradient does not specify the optimal step size, but only the optimal direction within an infinitesimal region.

CLIFFS AND EXPLODING GRADIENTS / 2

- Gradient clipping simply caps the step size to be small enough that it is less likely to go outside the region where the gradient indicates the direction of steepest descent.
- We simply “prune” the norm of the gradient at some threshold h :

$$\text{if } \|\nabla\theta\| > h : \nabla\theta \leftarrow \frac{h}{\|\nabla\theta\|} \nabla\theta$$

EXAMPLE: CLIFFS AND EXPLODING GRADIENTS



EXAMPLE: CLIFFS AND EXPLODING GRADIENTS / 2

Figure: “The objective function for highly nonlinear deep neural networks or for recurrent neural networks often contains sharp nonlinearities in parameter space resulting from the multiplication of several parameters. These nonlinearities give rise to very high derivatives in some places. When the parameters get close to such a cliff region, a gradient descent update can catapult the parameters very far, possibly losing most of the optimization work that had been done” (Goodfellow et al. (2016)).

REFERENCES



Ian Goodfellow, Yoshua Bengio and Aaron Courville (2016)

Deep Learning

<http://www.deeplearningbook.org/>



Yann Dauphin, Razvan Pascanu, Çağlar Gülçehre, Kyunghyun Cho, Surya Ganguli, Yoshua Bengio (2014)

Identifying and attacking the saddle point problem in high-dimensional non-convex optimization

<https://arxiv.org/abs/1406.2572>



Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, Tom Goldstein (2017)

Visualizing the Loss Landscape of Neural Nets

<https://arxiv.org/abs/1712.09913>



Konrad Koenigsberger (1997)

Analysis 2, Springer

REFERENCES / 2



Rong Ge (2016)

Escaping from Saddle Points

<http://www.offconvex.org/2016/03/22/saddlepoints/>

LICENSE & ATTRIBUTION

- This deck is adapted from **Introduction to Deep Learning (I2DL)**, Prof. Dr. Bernd Bischl, Statistical Learning and Data Science (SLDS), LMU Munich.
https://github.com/slds-lmu/lecture_i2dl
- Licensed **CC BY 4.0**:
<https://creativecommons.org/licenses/by/4.0/>
- Content has been merged, reordered, and consolidated from the original per-topic decks into this single session deck; not used verbatim.